Local Job Board Pipeline

This project is a daily job-search pipeline that fetches, scores, and summarizes job postings automatically, finishing
with a ranked digest to my inbox. It runs entirely on local infrastructure. The workflow is orchestrated by n8n, running
inside Docker, and all LLM inferences are handled by local models via Ollama, so there are no per-token API costs and
no data leaves the machine.

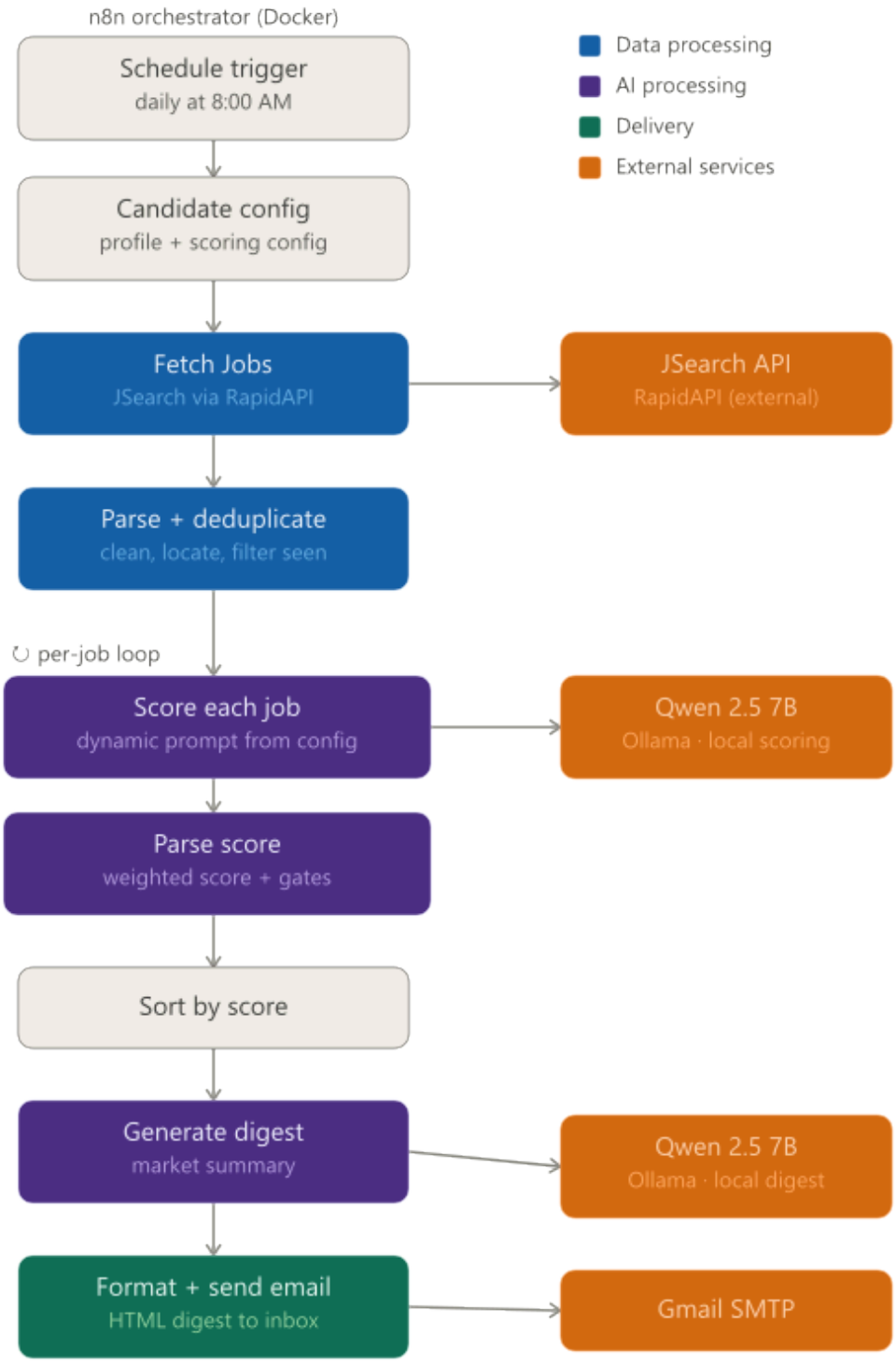
The pipeline is deliberately split between deterministic logic and LLM judgment. Location fit and seniority classification
are computed from a centralized candidate profile before any model calls are made. It was found that the models performed
poorly at these tasks, particularly location fit, and the detrministic approach is simple. The models are responsible for
tasks they are well suited for: reading job descriptions, assessing skill match against a rubric, and writing concrete
reasonings based on the postings actual requirements. Final scores are computed as a weighted average of five dimensions:
skill match, seniority fit, location fit, growth signal, and compensation fit. There are hard gates in place for
disqualifing roles that are independant of score.

The candidate profile, search parameters, scoring weights, city tier lists, seniority markers, and model names all live
in a single `candidate_profile.json` config file. In this way, fine-tuning requires only a single change in one place.

Architecture

The pipeline runs on a daily schedule inside a self-hosted n8n instance. All LLMinference goes to Ollama on the host machine,
reached from the container via `host.docker.internal`. The only outbound network calls are to the JSearch API, which offers

200 requests per month in their free tier, and Gmail's SMTP server for delivery.



Pipeline Stages

Candidate Config

This node runs as soon as the schedule is triggered. It is a Code node that returns the candidate profile in structured JSON. It contains search parameters, target roles, skills lists, location tier maps, seniority markers, scoring weights, and model names. Every downstream node that needs any of these data can read it from this node by name rather than maintaining its own hardcoded data.

Fetch Jobs

A GET request is sent to the JSearch `/search-v2` endpoint via rapidAPI

Design Decisions

Configuration

Setup